

Lab Manual

Mobile Application Development

Lab: CardView with RecyclerView in Android Studio

Subject: Mobile App Development

Tool Used: Android Studio (Java)

Team Members:

Adina Abrar

2023-ag-9487

Ume Aiman Shahzad

2023-ag-9691

Submitted To: Miss Wajeeha Azmat

Date: 31 May, 2026

Course Code: CS-512

Objective

The main purpose of this lab is to learn how to use CardView and RecyclerView in Android Studio. In this assignment we made an app which shows a list of programming languages. Each programming language is shown in a card which has its logo image, name, and a short description.

By doing this lab, we learned how to:

- Create a new project in Android Studio
- Add RecyclerView in the layout
- Make a custom card layout (card_item.xml)
- Create a Model class to store data
- Create a Holder class to hold the views
- Create an Adapter class to connect data with RecyclerView
- Display final output on emulator

Tools and Technologies Used

IDE: Android Studio

Language: Java

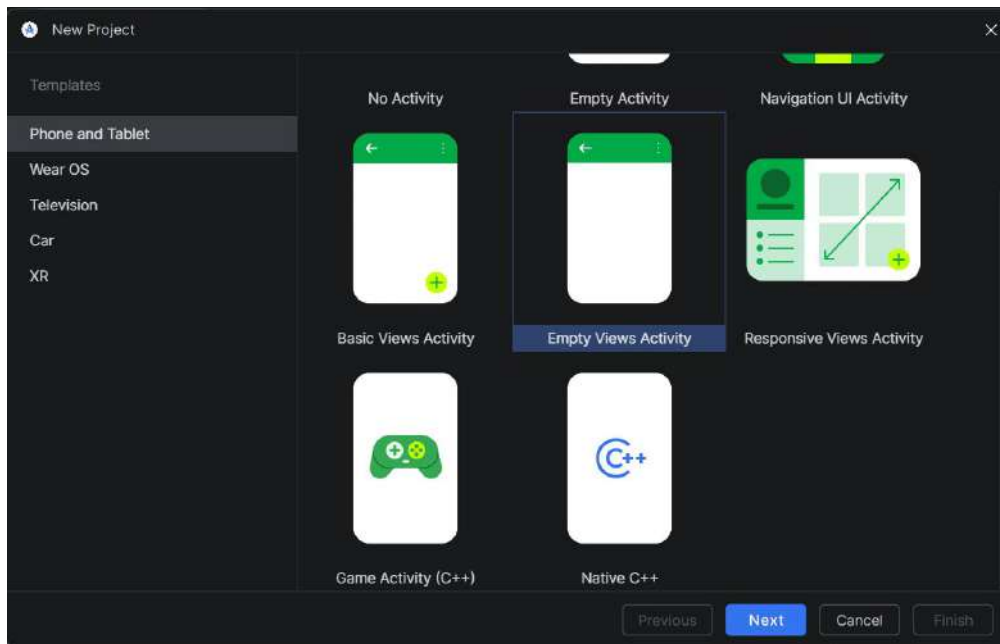
Min SDK: API 24 (Android 7.0 - Nougat)

Build Config: Kotlin DSL (build.gradle.kts)

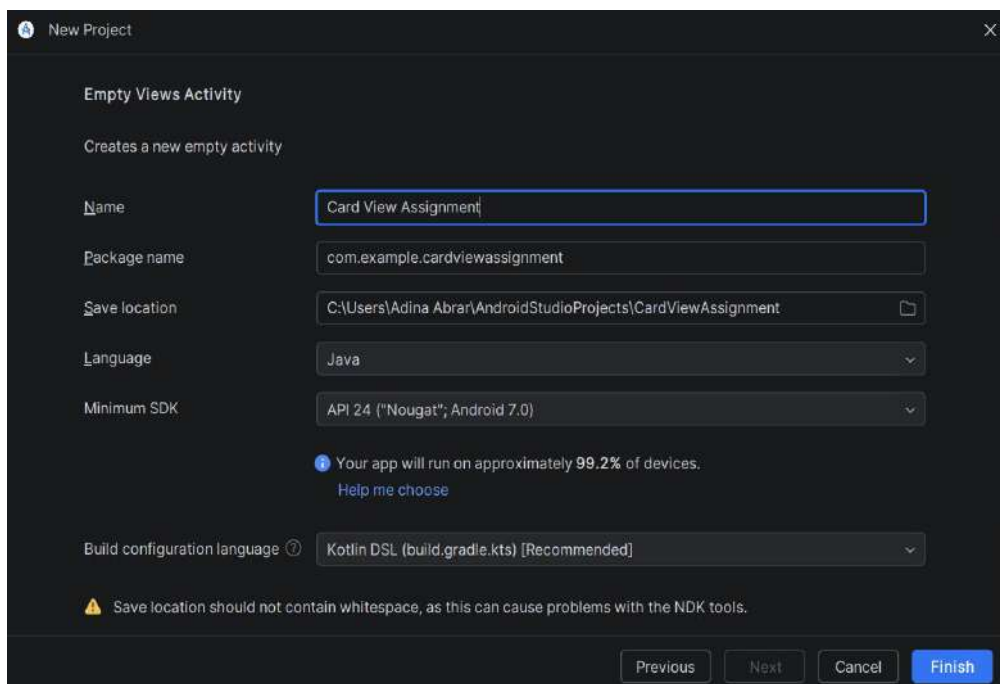
RecyclerView widget was used for showing the list, and CardView was used to make each list item look like a nice card.

Step 1: Create New Project

First we opened Android Studio and clicked on "New Project". A window appeared asking us to choose a template. We selected "Empty Views Activity" from the Phone and Tablet section because we want to start with a simple, clean screen.



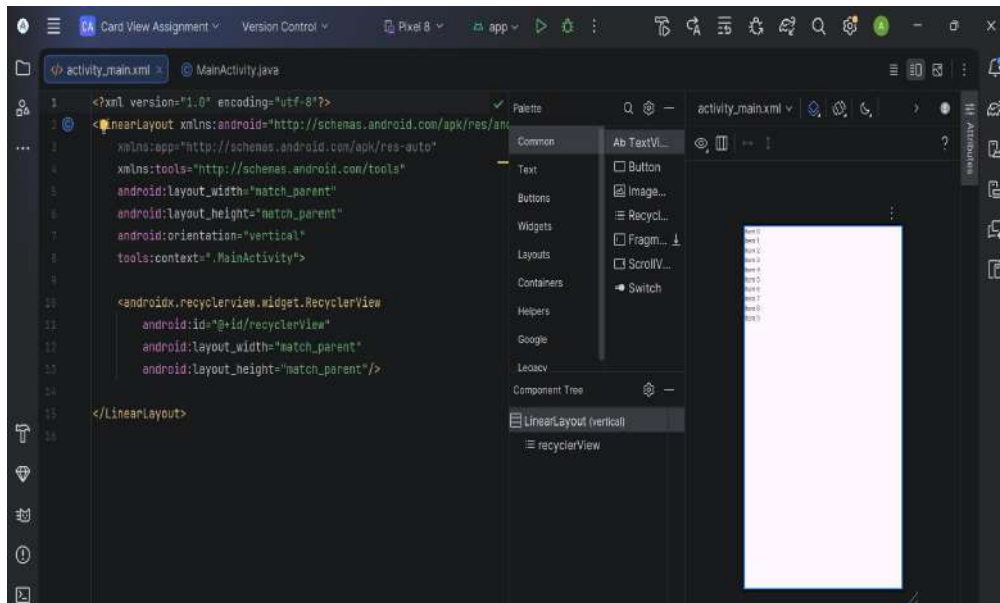
After selecting the template, we clicked Next. Now the project configuration screen appeared. We gave the project name "Card View Assignment", selected Java as the language, and kept the Minimum SDK as API 24. Then clicked Finish.



Step 2: Add RecyclerView in activity_main.xml

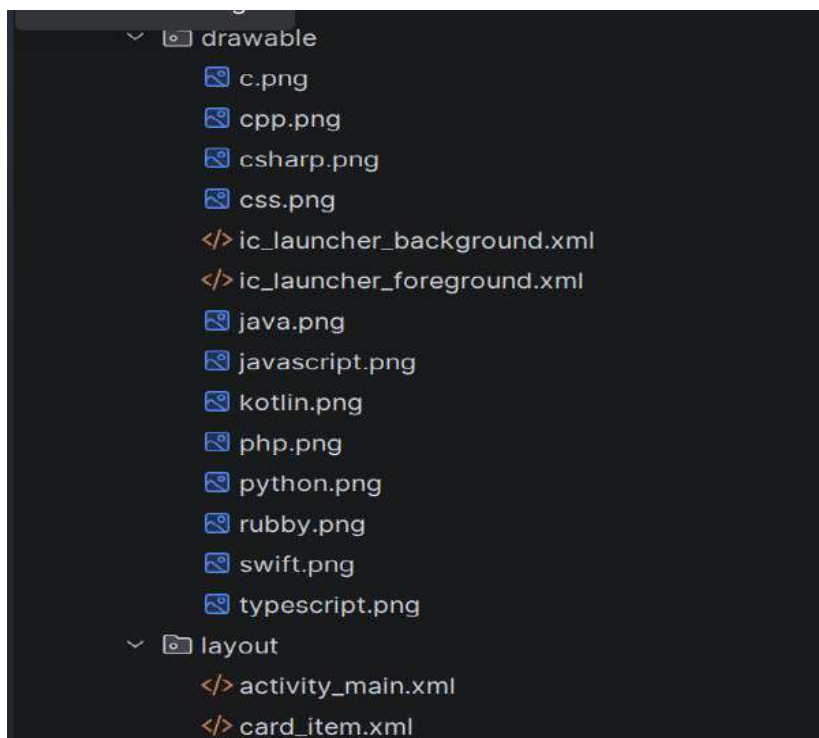
After the project was created, we opened the activity_main.xml file. We deleted the default TextView and added a RecyclerView widget. The RecyclerView has id

"recyclerView" and its width and height are set to match_parent so it takes the full screen.



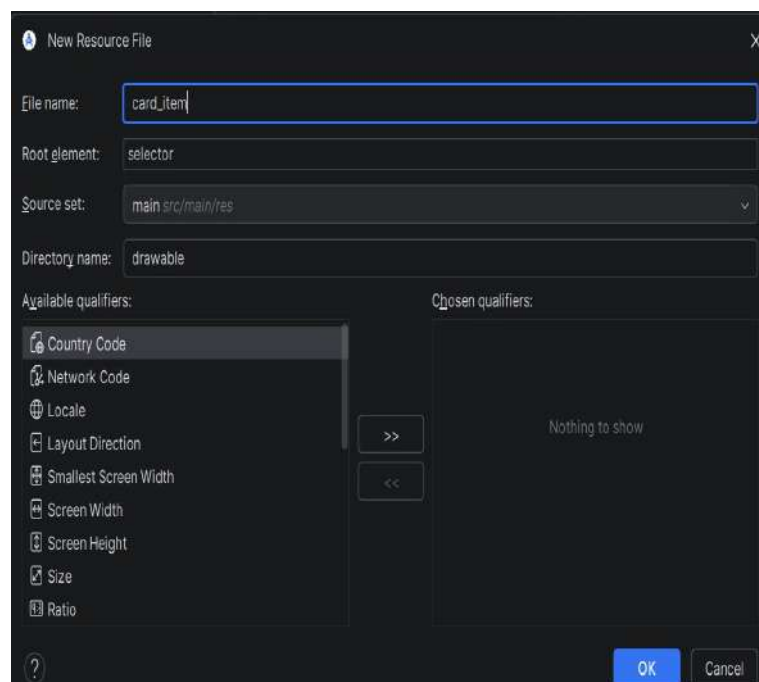
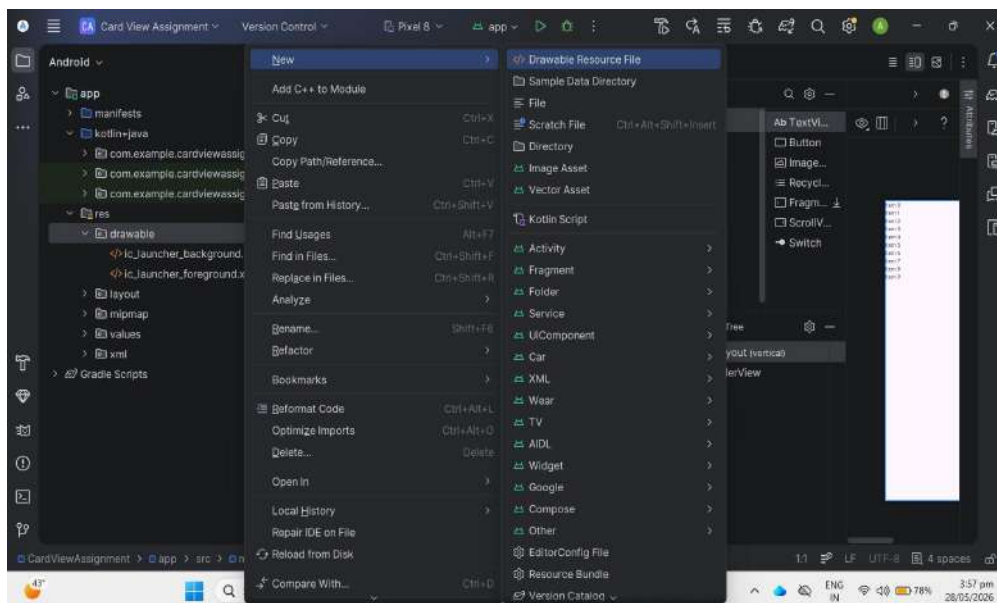
Step 3: Add Images to Drawable Folder

We needed images for each programming language card. So we right-clicked on the "drawable" folder, then went to New > Drawable Resource File. We added PNG images for Python, Java, Kotlin, JavaScript, PHP, C++, Swift, Ruby, and other languages.



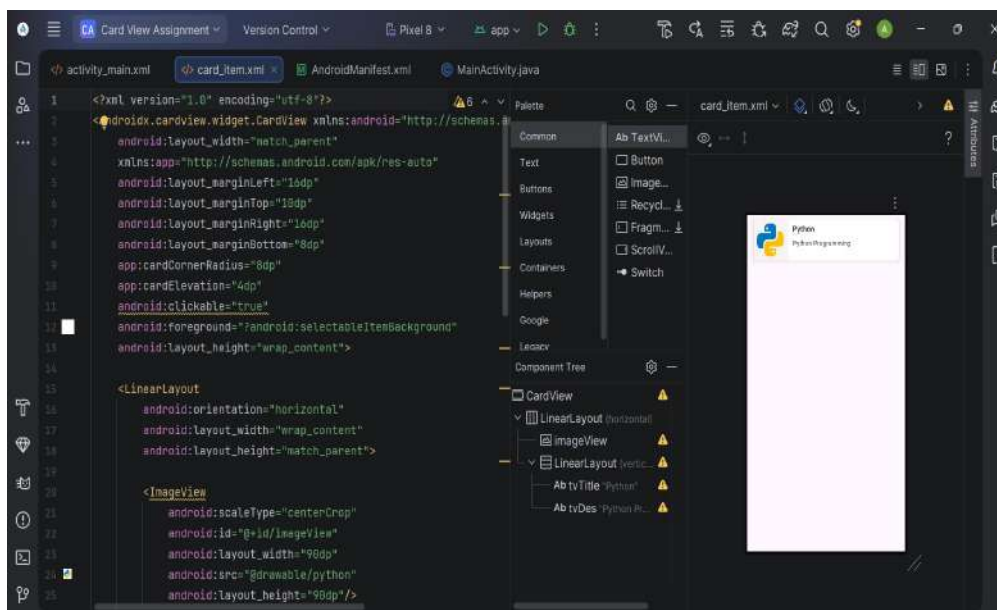
Step 4: Create card_item.xml Layout

Now we created a new XML layout file for the card design. We right-clicked on the "layout" folder and created a file named "card_item.xml". In this file we added a `CardView` as the root element. Inside it we put a horizontal `LinearLayout` with an `ImageView` (for the logo) and another vertical `LinearLayout` (for title and description text).



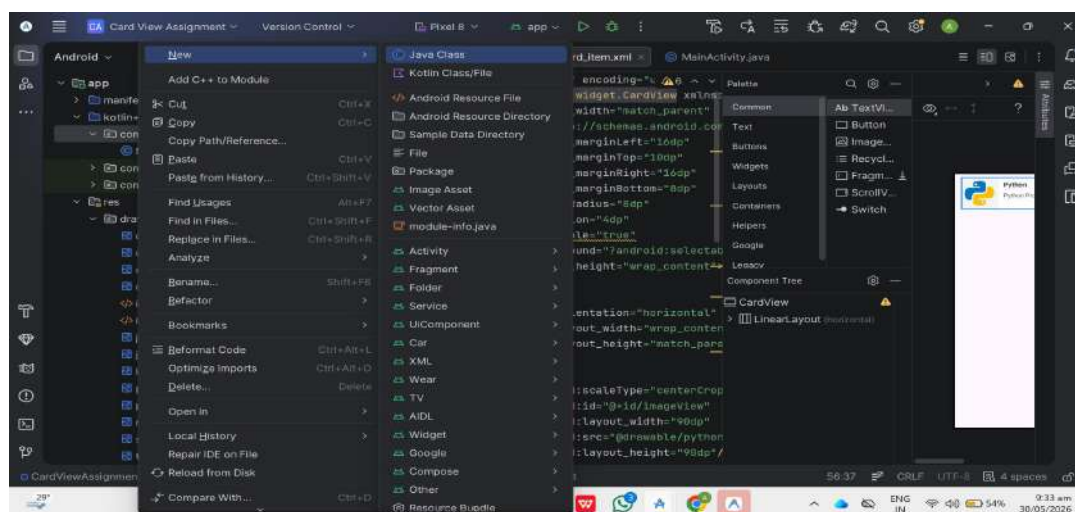
Step 5: card_item.xml Layout

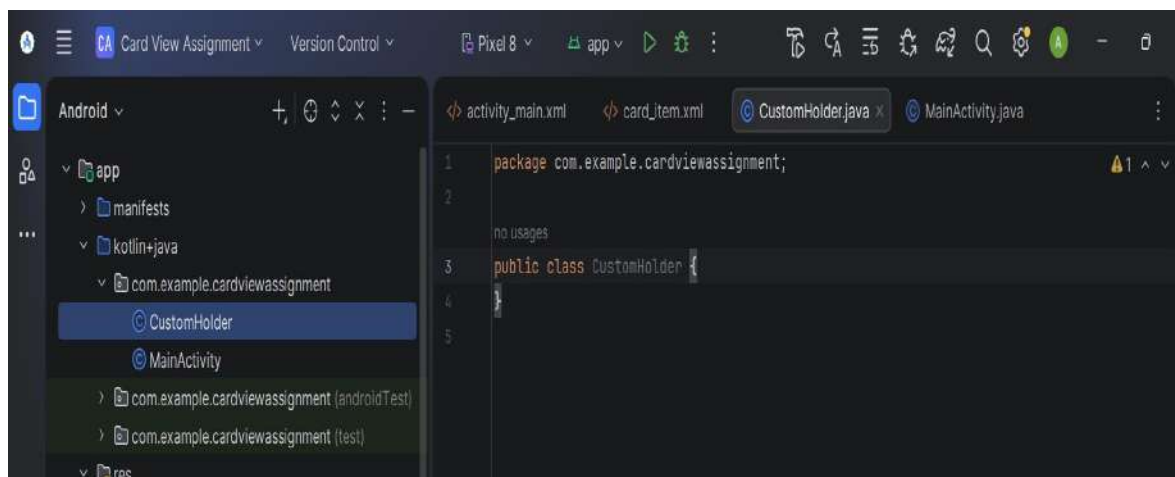
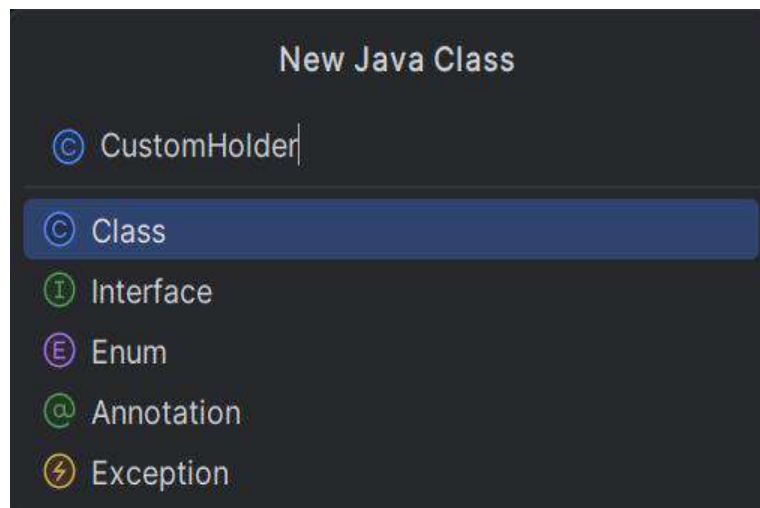
The root element we used is `CardView` which automatically gives rounded corners and shadow to the card. Inside the `CardView` we added a horizontal `LinearLayout`. On the left side we placed an `ImageView` to show the logo of the programming language. On the right side we added a vertical `LinearLayout` which has two `TextView`s inside it — one `TextView` is for the language name and the other one is for the short description. After writing the XML code, we could see the card preview on the right side of Android Studio which was showing a Python card.



Step 6: Create CustomHolder.java Class

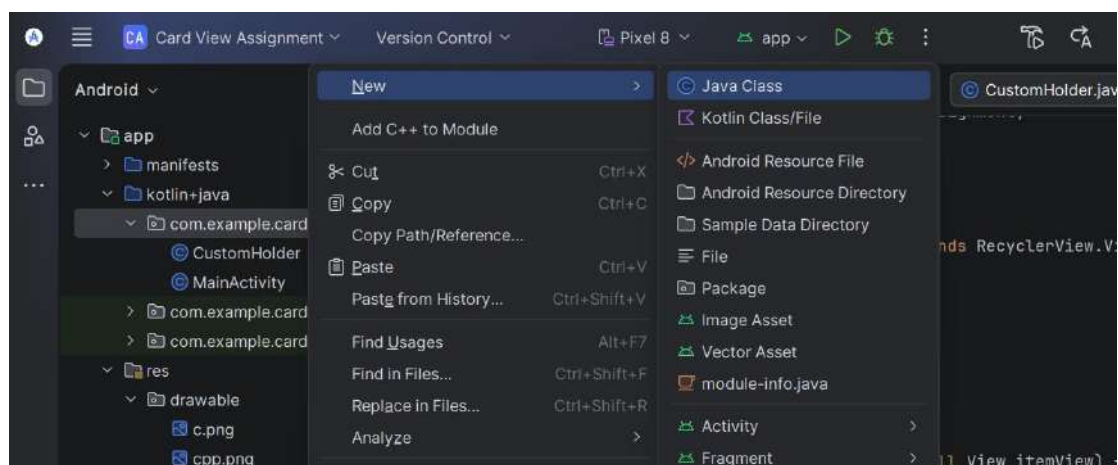
We needed a `ViewHolder` class to hold the references to all the views inside each card. We right-clicked on the package name > New > Java Class. We named it "CustomHolder". This class extends `RecyclerView.ViewHolder`. Inside it we declared the `ImageView` and the two `TextView`s, and initialized them in the constructor.

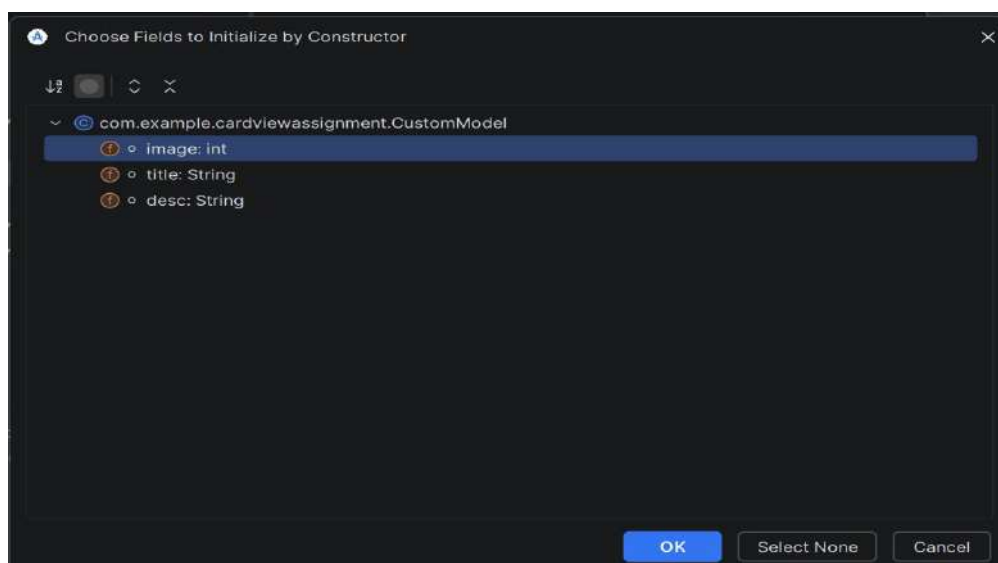
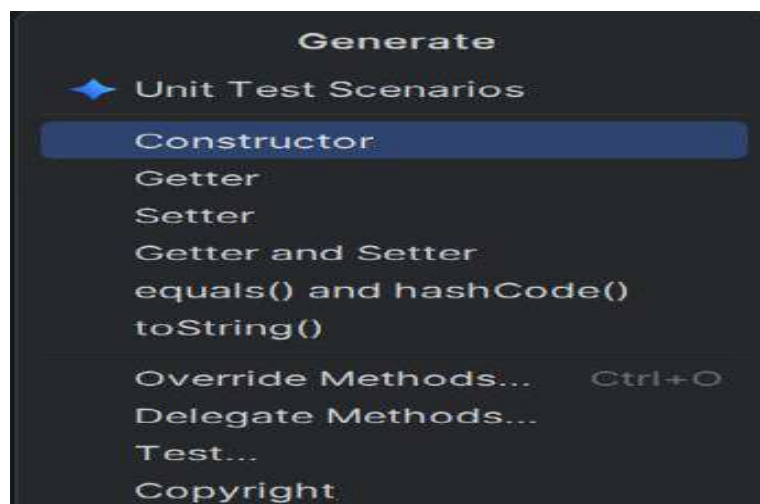
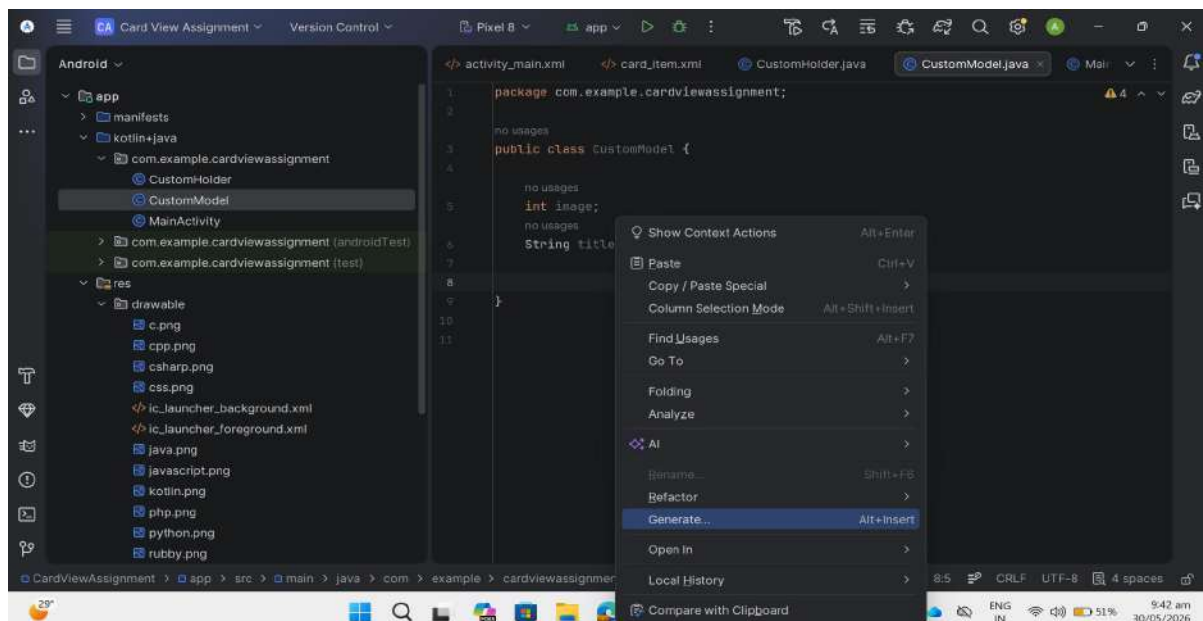




Step 7: Create CustomModel.java Class

We made another Java class named "CustomModel". This class is used to store the data for each card item. It has three fields: image (int), title (String), and desc (String). We used Android Studio's Generate feature to automatically create the constructor, and getters and setters for all fields.






```

no usages
9  public CustomModel(int image, String title, String desc) {
10      this.image = image;
11      this.title = title;
12      this.desc = desc;
13  }
14  }

```

Step 8: Creating Getters and Setters for CustomModel

In this step we generated the getters and setters for the CustomModel class using Android Studio's Generate feature. We right-clicked inside the class, went to Generate > Getter and Setter. Android Studio automatically created all the get and set methods for all three fields.



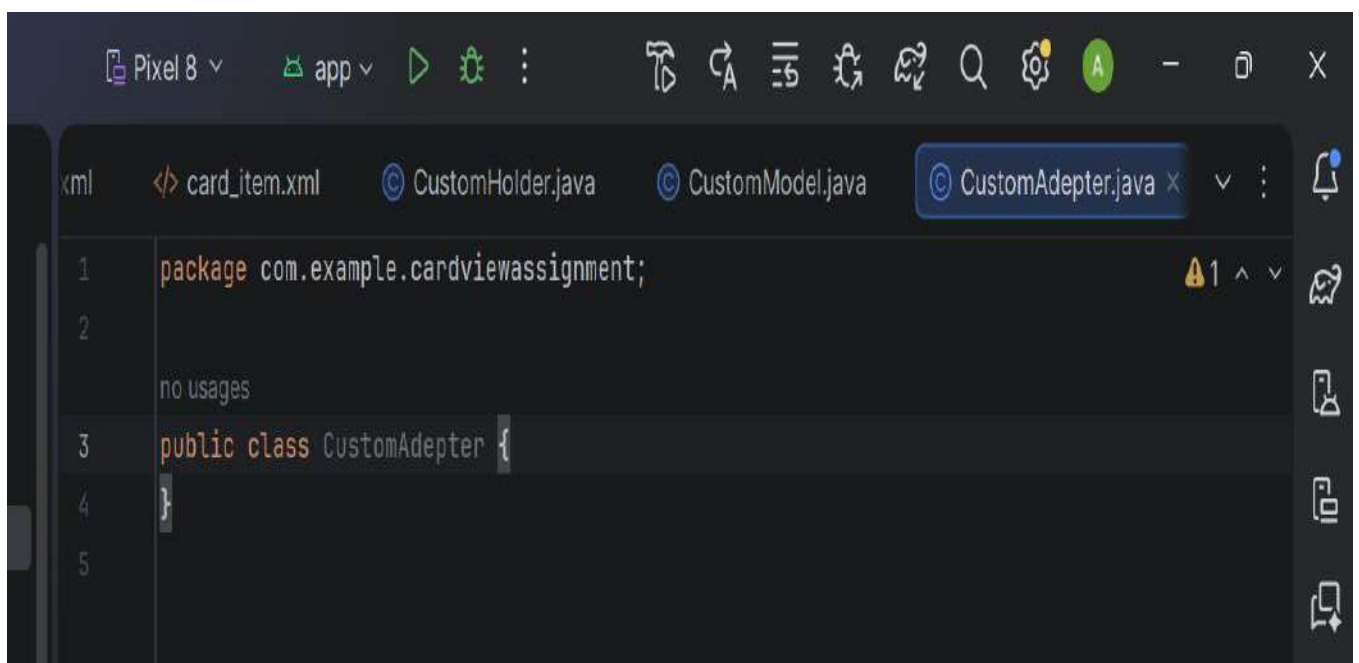
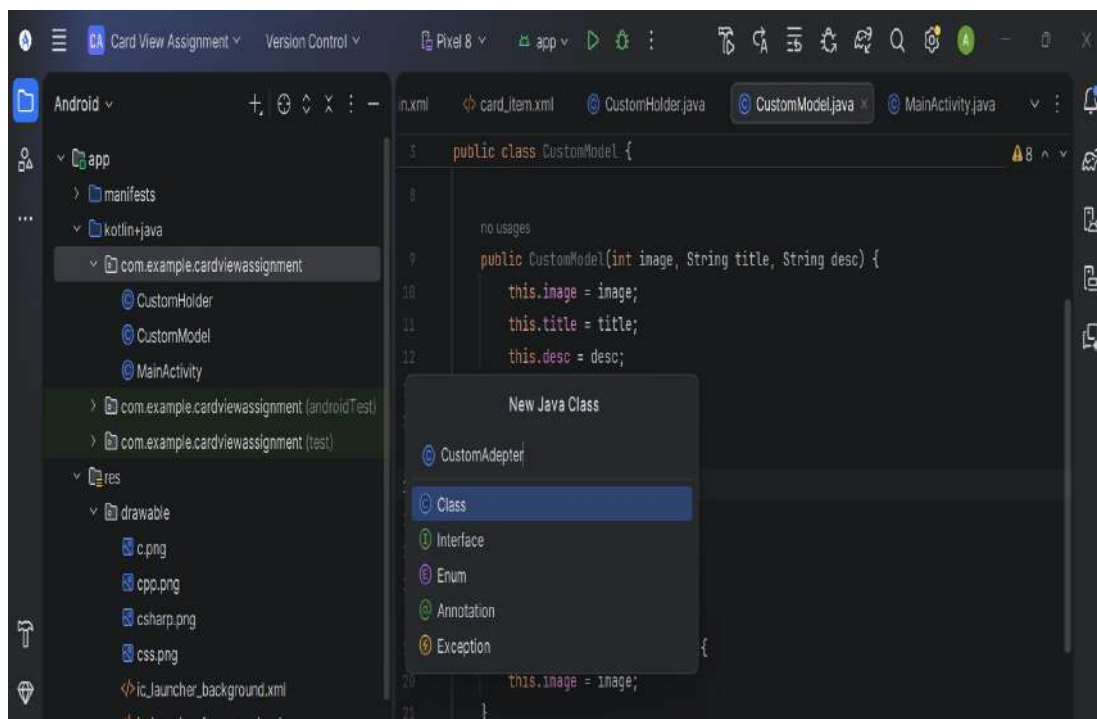
```

3  public class CustomModel {
4      no usages
5      public int getImage() {
6          return image;
7      }
8
9      no usages
10     public void setImage(int image) {
11         this.image = image;
12     }
13
14     no usages
15     public String getTitle() {
16         return title;
17     }
18
19     no usages
20     public void setTitle(String title) {
21         this.title = title;
22     }
23
24     no usages
25     public String getDesc() {
26         return desc;
27     }
28 }

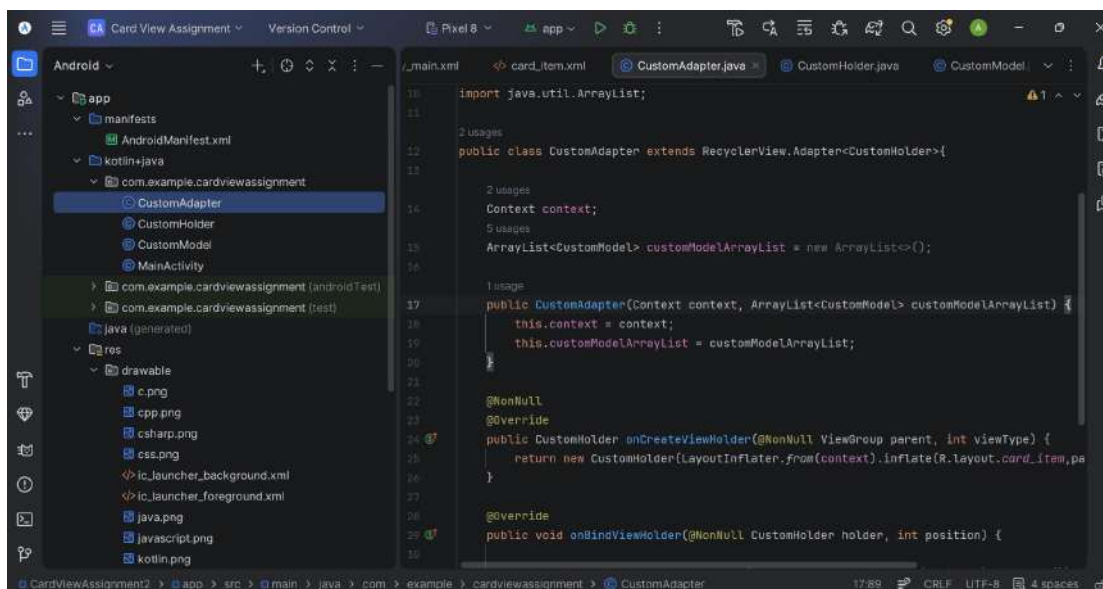
```

Step 9: Creating CustomAdapter Class

In the last coding step we opened MainActivity.java. Here we first found the RecyclerView using its id. Then we set a LinearLayoutManager on it (so cards appear one below another in a vertical list). After that we created an ArrayList and added CustomModel objects to it - one for each programming language with its image, title, and description. Finally we created a CustomAdapter object and set it on the RecyclerView.



In this step we created a new Java class named "CustomAdapter". We right-clicked on the package name, went to New > Java Class, typed "CustomAdapter" and selected Class. This class extends RecyclerView.Adapter and it works as a bridge between our data list and the RecyclerView. Inside this class we wrote three main methods. The first method is onCreateViewHolder() which inflates the card_item.xml layout and returns a CustomHolder object. The second method is onBindViewHolder() which takes the data from the ArrayList and sets it into the ImageView and TextViews of each card using the CustomHolder. The third method is getItemCount() which simply returns the total number of items in the list so RecyclerView knows how many cards to show.



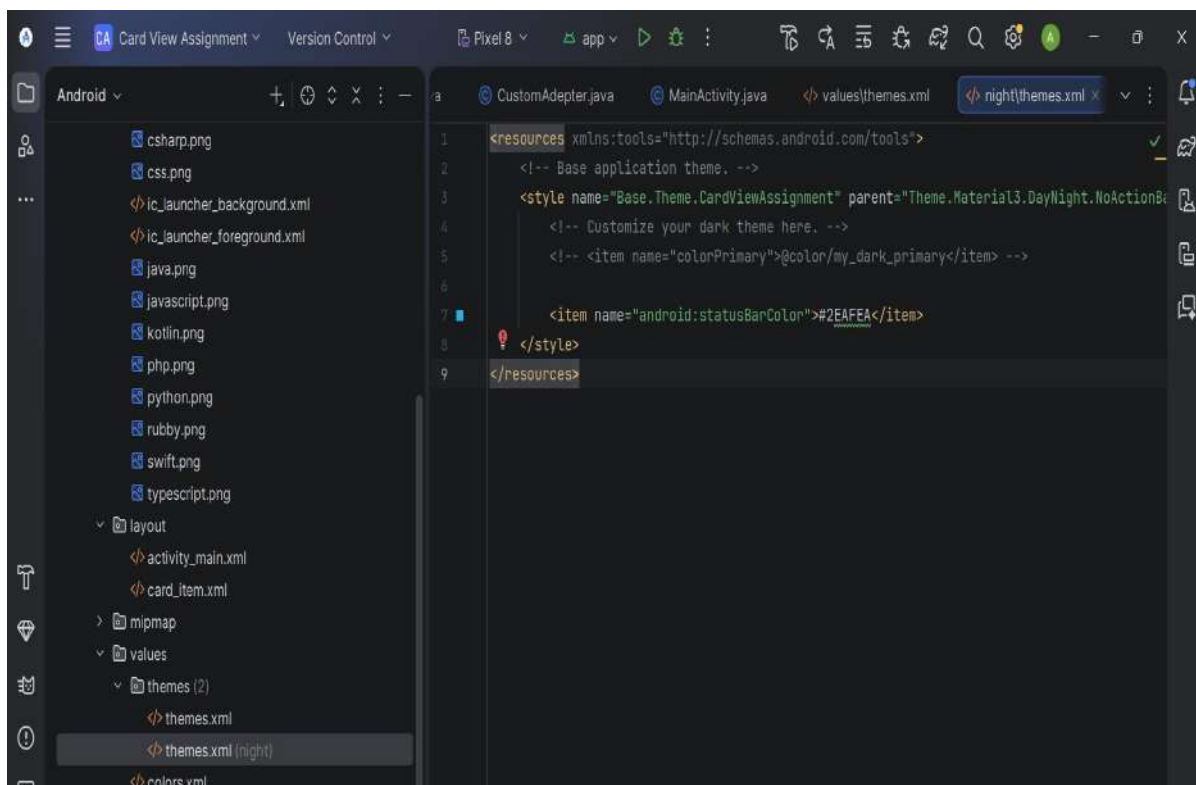
Step 10: Adding Language Data and Setting Up RecyclerView in MainActivity.java

In this step we opened MainActivity.java and wrote the main logic of the app. First we found the RecyclerView using findViewById and its id "recyclerView". Then we set a LinearLayoutManager on it so the cards appear one below the other in a vertical list. After that we created an ArrayList of CustomModel objects. We added one CustomModel object for each programming language — Python, Java, Kotlin, JavaScript, PHP, C++, Swift, and Ruby. Each object was given three things — the drawable image of that language, the name of the language, and a short description. At the end we created a CustomAdapter object by passing the context and the ArrayList to it, and then we set that adapter on the RecyclerView. After this the app was ready to show all the language cards on the screen.

```
20 setContentView(R.layout.activity_main);
21 recyclerView = findViewById(R.id.recyclerView);
22 recyclerView.setLayoutManager(new LinearLayoutManager(context, this));
23
24
25 customModelArrayList.add(new CustomModel(R.drawable.python, title: "Python",
26 customModelArrayList.add(new CustomModel(R.drawable.java, title: "Java",
27 customModelArrayList.add(new CustomModel(R.drawable.kotlin, title: "Kotlin",
28 customModelArrayList.add(new CustomModel(R.drawable.javascript, title: "Javascript",
29 customModelArrayList.add(new CustomModel(R.drawable.php, title: "PHP",
30 customModelArrayList.add(new CustomModel(R.drawable.cpp, title: "CPP",
31 customModelArrayList.add(new CustomModel(R.drawable.swift, title: "Swift",
32 customModelArrayList.add(new CustomModel(R.drawable.ruby, title: "Ruby",
33 CustomAdepter customAdapter = new CustomAdepter(context, this, customModelArrayList);
34 recyclerView.setAdapter(customAdapter);
35
36 }
37 }
```

Step 11: Changing the Theme of the App

In this step we changed the theme of the app to make it look better. We opened the themes.xml file from the values folder and changed the base theme to a dark theme. We also changed the status bar color by setting the colorPrimary to a dark color code. This made the top bar of the app look dark and more professional. The night theme file themes.xml (night) was also updated so the app looks consistent in both light and dark mode.



```
1 <resources xmlns:tools="http://schemas.android.com/tools">
2   <!-- Base application theme. -->
3   <style name="Base.Theme.CardViewAssignment" parent="Theme.Material3.DayNight.NoActionBar">
4     <!-- Customize your dark theme here. -->
5     <!-- <item name="colorPrimary">@color/my_dark_primary</item> -->
6
7     <item name="android:statusBarColor">#2EAFEA</item>
8   </style>
9 </resources>
```

Output

After writing all the code we ran the app on mobile. The app opened successfully and showed a scrollable list of programming language cards

